

A Detailed Analysis of the LLVM's Register Allocators

Tiago Cariolano de Souza Xavier, George Souza Oliveira, Ewerton Daniel de Lima and Anderson Faustino da Silva
State University of Maringá – Brazil

Email: {tiago.cariolano, geo.soliveira, ewertondanieldelima}@gmail.com, anderson@din.uem.br

Abstract—The goal of a register allocation is to allocate a unbounded amount of program values to a finite amount of machine registers, which is an issue that impacts the performance of the final code. Thus, a good register allocator has the goal of minimizing the amount of program values that should be represented in memory. This paper presents a detailed analysis of the register allocators provided by LLVM framework, namely: *fast*, *basic*, *greedy*, and *PBQP*. The analysis was conducted to evaluate runtime, compile time and the amount of spills. Furthermore, hardware performance counters were analyzed in order to describe in more detail the behavior of the register allocators. The results demonstrated that *greedy* register allocator is the best one for minimizing the runtime and the amount of spills, otherwise the *basic* and *fast* are the best choice for compile time.

Keywords—LLVM; Register Allocator; Performance.

I. INTRODUCTION

The main goal of a traditional compiler is generating code that results in a fast runtime, and if possible in a low compile time [1]. These goals can be addressed in different ways; developers can provide in their compilers different optimizations, and/or they can improve one or more compiler phases.

One of the most important phases in compiler design, which directly affects the performance of the final code is register allocation [2]. The goal of this phase is to decide which program values should be kept in registers or represented in memory during program execution. Register operations are faster than memory operations, thus the register allocation will try to maximize the amount of values that must be kept in registers [3].

However, in practice, the registers are a scarce machine resource [4], despite they are the fastest computational units. Therefore, the problem in this context is to minimize the movement of data between registers and memory hierarchy.

Another issue in the context of register allocation is related to how the programs use the registers. Some architectures use some registers for specific purposes and therefore some conventions need to be observed [5]. For example, on x86 machines, SI and DI registers are general purpose registers, but the convention is that they are used as pointers for string operations. Besides, the EP and SP registers also are classified as general purposes registers, but they are conventionally used for manipulating the stack. These restrictions make the register allocation problem even more complex.

Several researches have proposed different register allocators, such as: allocators based on graph coloring [6] [7] [8],

allocators that perform a linear scan in the code [9], [10], [11], [12], allocators based on metaheuristics [13] [14], among others. Some of these allocators are currently available on various compiler infrastructures, such as ROSE [15], FIRM [16] and LLVM [17]. Therefore, understanding the register allocator performance is an important issue to be addressed.

In this context, this paper describes a detailed analysis of the LLVM register allocators, namely: *fast*, *basic*, *greedy* and *PBQP*. The motivation for this analysis is to provide information for future compiler implementations. Besides, the evaluation of the different strategies of register allocation can assist the user to decide which allocator to use in different contexts. It is also useful to know in which context a specific allocator should be used. If some research is based on JIT compilation [18], [19], [20], an goal will be decrease the compile time. Therefore, in this context, it may be more convenient to use an allocator that runs fast.

In this paper, the evaluation demonstrated that the *greedy* allocator is the most efficient LLVM allocator. This allocator has better performance than others, in the amount of spills and also the program runtime. But, if the goal is to minimize the compile time the best choice is to use *basic* or *fast* register allocators.

This article is organized as follows. The Section II presents a brief discussion on register allocation. The Section III presents the LLVM register allocators. The Section IV presents a detailed evaluation of the LLVM register allocators. And finally, the Section V presents our concluding remarks.

II. REGISTER ALLOCATION

Register allocation is one of the most important compiler optimizations affecting the performance of compiled code [2]. It determines which of the program values (variables and temporaries) should be kept in machine registers (or memory) during the execution. In a real machine, registers are usually few and fast to access [4], [21], so the problem addressed here is how to minimize the traffic between registers and memory hierarchy. Therefore, the challenge is to relegate the least program values to memory.

Register allocation even though can be mapped as a graph coloring problem (GCP) [6], [22], a highly effective approach to intraprocedural register allocation, it has a slight variation: it is forced to eliminate conflicting edges besides coloring the graph with just k colors (registers). A graph coloring

register allocation can be briefly described as follows. During code generation, the compiler [1] uses infinite symbolic registers to hold program values, and determines what values are candidates for allocation to registers. First, the register allocator generates a so-called interference graph [2], whose vertices/nodes represent the program values and the machine real registers and whose edges represent interferences. In this graph, an edge (interference) is added either if two values are simultaneously live or a value cannot be or should not be allocated to that register. After that, it will color the interference graph nodes with k colors, so that any two adjacent nodes have different colors. And, finally, the allocator will allocate each value to the register that has the same color that was assigned to it.

Graph coloring register allocation (GCRA) was proposed by Chaitin *et al.* [6], [23]. This allocator was used in an experimental IBM 370 PL/I compiler. Currently, versions of it and allocators derived from it have been used in mainstream compilers. Subsequently, several works added improvements to Chaitin's allocator [24], [25]. The most successful design for GCRA was developed by Briggs *et al.* [26]. Their work redesigned the Chaitin *et al.* allocator to delay spill decisions until later on in the allocation process. Runeson and Nyström proposed a generalization of Chaitin's allocator, which allows it to be used for irregular architectures [27]. This work is an interesting framework for a retargetable graph-coloring allocator.

George and Appel [5], [8] designed a GCRA that interleaves Chaitin-style simplification steps with Briggs-style conservative coalescing. They ensure this approach eliminates more move instructions than Briggs's one, while still guaranteeing not to introduce spills.

Daveou *et al.* [28] presented a register allocation framework designed to address the embedded processor specificities, such as smaller number of registers, irregular and constrained register sets, and instructions operating on short or long data types. This allocator is based on Briggs's one, with two new components developed to improve performance, namely: a spill manager that optimizes spill operations, and a code manager that optimizes the move operations inserted by the allocator.

A problem with some GCRA approaches is the fact that some of them apply simple heuristic methods resulting often in a poor allocation. In this case, there will be constant data traffic between processor and memory causing a performance loss. To address this issue, several works proposed the use of metaheuristics with the goal of using a more aggressive strategy of graph coloring [29], [30], [31], [32].

Mahajan and Ali [29] developed a heuristic algorithm for GCRA for embedded processors, based on hybrid evolutionary algorithm that uses a new crossover operator and a new local search. The assumption of the authors is that traditional register allocators are developed to homogenous register set, but embedded processors need special attention due to its irregularities.

The works of Shamizi and Lotfi [30], and Topcuoglu *et al.*

[31] also developed a register allocator based on hybrid evolutionary algorithm. Similar to work of Mahajan and Ali, these works also proposed crossover operators, and the use of local search. And Wu and Li [32] proposed a hybrid metaheuristic algorithm for GCRA that combines several ideas from classic GCRA algorithms, besides evolutionary algorithms [33] and Tabu Search [34], [35]. The main idea of this approach is to exploit the interplay between intensification and diversification of the solution space. The authors argue it is a good solution to prevent searching processes from cycling, i.e., from endlessly revisiting the same solutions set, besides can impart additional robustness to the search.

In applications where compilation time is a concern, such as dynamic compilation systems [18], [19], [20], researchers try to balance the time necessary for allocation against the resulting code quality. And, in this context, they do not choose a register allocation algorithm based on graph coloring, because it is a complex algorithm, besides being a time-consuming register allocator, but based on linear scan or other strategy [9], [10], [11], [12].

Several researches also analysis several register allocation strategies. Cooper *et al* [36] conducted an analysis of two register allocation strategies based on graph coloring: the Chaitin-Briggs and Callahan Koblenz allocators. The work of Cooper *et al* evaluated only the amount of spills and copy elimination. Pereira and Palsberg [37] proposed a new register allocator based on solving puzzles. To compare the results of their new allocator, Pereira and Palsberg held a comparison of it with the LLVM register allocators. Cai *et al* [38] also evaluated different register allocation strategies. The evaluation analyzed two strategies, namely: linear-scan, and graph coloring. This work evaluated the performance based on runtime and amount of spills.

III. LLVM REGISTER ALLOCATORS

The LLVM is a compiler infrastructure that the goal is to be a long-life analyses and code transformations infrastructure [17]. The intermediate representation provides several novel features, which enables both source code analyses and code transformations. Moreover, this intermediate representation provides facilities for developing new compilers, besides compilation strategies [39].

LLVM infrastructure provides four register allocator, namely: `fast`, `basic`, `greedy` and `PBQP`. `Fast` allocator performs the task of assigning values to registers at the level of basic blocks, its behavior consists in tracing sequential code and allocates each value found in any available register. `Basic` is an extension of the `fast`, except that `basic` is based on an incremental approach, using a heuristic strategy `basic` assigns each live range, one by one, to registers. `Greedy` is an implementation of the `basic` allocator that uses global live range. Finally, `PBQP` is an algorithm that transforms the problem of register allocation into Partitioned Boolean Quadratic Problem. The next four sections describe in more details each register allocator.

A. The Fast Register Allocator

This allocator is a local register allocator, which has the simplest strategy when compared with the others. It scans the program linearly and assigns values to registers as they appear. This scanning is performed at the level of basic blocks, so there are no gaps between basic blocks. When a register is in use, the value is spilled to memory, however spills always happen at the end of the basic blocks. This allocator is used for debugging purpose in LLVM.

B. The Basic Register Allocator

The *basic* register allocator is an extension of the linear scan register allocator proposed by Poletto [9] with some extensions. The linear scan performs the register allocation via a simple approach that does not need to create the traditional interference graph. The algorithm visits all live ranges of values in a linear order and those that are alive are maintained in an active list. This list, which replaces the interference graph, is used to detect the interference between the live ranges. The allocator assigns to each available physical register a range and when all registers are blocked, an interval is chosen to be spilled into memory.

In LLVM this allocator works as a code rewriter, rewriting virtual addresses to physical addresses. However, as the linear scan it performs spill without a previous division of the life intervals. Besides, the rewriter eliminates unnecessary code. For example, if the register allocator recharges from a same stack slot twice, the rewriter performs the reuse of a register eliminating the second reload.

Due to the use of simple data structures and a linear scan in the code, *basic* is a fast algorithm that enables several improvements to increase the performance of the final code, as the use of a rewriter to eliminate instructions. However, its disadvantage is to perform spill before to try the opportunity of splitting the intervals into smaller pieces, and consequently minimize the register pressure. An alternative to solve this problem would be to perform a backtrack, but its process can be time consuming that would cause the loss of speed.

In fact, *basic* improves the performance of a traditional linear scan register allocator, because *basic* is able to search for life intervals in any order, not necessarily from the beginning to the end. Due to instead of using an active list to check for interference intervals, *basic* uses a set of life interval unions.

The life intervals have a spill weight and this weight is used to order them decreasingly. This order defines a priority queue and consequently which ranges will be allocated first. This weight value is the density of its use, thus intervals used very often possess high weights. Intervals that are unions of intervals have the highest spill weight, which causes them to be allocated to the first register. Besides, the ranges created by spill code have infinite weight, because they can not be reassigned. When a range can not be assigned to a physical register it is spilled.

C. The Greedy Register Allocator

The greedy allocator uses a similar strategy to that used by *basic*. Greedy also uses a priority queue and life interval unions, but with substantial differences.

Due to the heuristics used by *basic* allocator be based on spill weight, which represents the density of its use, the smallest intervals will be allocated first. This causes the spill of the largest intervals. The greedy addresses this problem providing registers first for large intervals. Thus, the large ranges are allocated and the minors fill the gaps.

However, another problem occurs when there are a lot of large intervals, due to the fact that small intervals with high spill weight will be spilled, which is also undesirable. To solve this problem the greedy performs an spill operation of its physical registers. After that, the physical registers come back to the priority queue and can be reused.

D. The PBQP Register Allocator

The PBQP register allocator is the LLVM register allocator that performs allocation based on the Partitioned Boolean Quadratic Programming [40]. In general, the algorithm maps the problem of assigning values to physical registers for a PBQP scheme, it solves this problem and maps the solution back to the context of register allocation.

The PBQP is a variation of the Quadratic Assignment Problem (QAP) [41] and the formal definition of this problem is as follow. Consider a set of discrete variables $X = x_1, x_2, \dots, x_n$ with finite domains $D = D_1, D_2, \dots, D_n$ where $m_i = |D_i|$ is the domain size (the amount of elements) D_i . There is a function $h : X \rightarrow \mathcal{D}$, where $\mathcal{D} = D_1 \cup D_2 \cup \dots \cup D_n$, which corresponding to assigning a variable x_i for some element d_i of $\mathcal{D}$. Each assignment of a variable x_i to d_i element has a cost given by the function $c(x_i, d_i)$ and each assignment of two related variables x_i and x_j to elements d_i and d_j , respectively, have a cost given by the function related to $C(x_i, x_j, d_i, d_j)$. Therefore, the total cost of a solution h is

$$f = \sum_{1 \leq i \leq n} c(x_i, h(x_i)) + \sum_{1 \leq i < j \leq n} C(x_i, x_j, h(x_i), h(x_j))$$

The solution to the problem is to find an assignment that minimizes this function.

The PBQP can also be represented by a graph $G = (V, E)$, where the nodes are discrete variables and the edge (i, j) for $i < j$ belongs to E if C_{ij} is nonzero, where C_{ij} is a matrix which each of its elements corresponds to an assignment (x_i, x_j) to (d_i, d_j) . The mapping of register allocation problem to PBQP problem is done mapping the interference graph for the PBQP graph.

IV. THE LLVM REGISTER ALLOCATORS PERFORMANCE

In order to evaluate the LLVM register allocators were conducted several experiments on an Intel Xeon E5504 2.0 GHz, 24 GBytes of RAM, running Ubuntu 12 kernel 3.2.0_24 X86_64. The experiments were performed for several benchmarks, and each benchmark program was executed ten times.

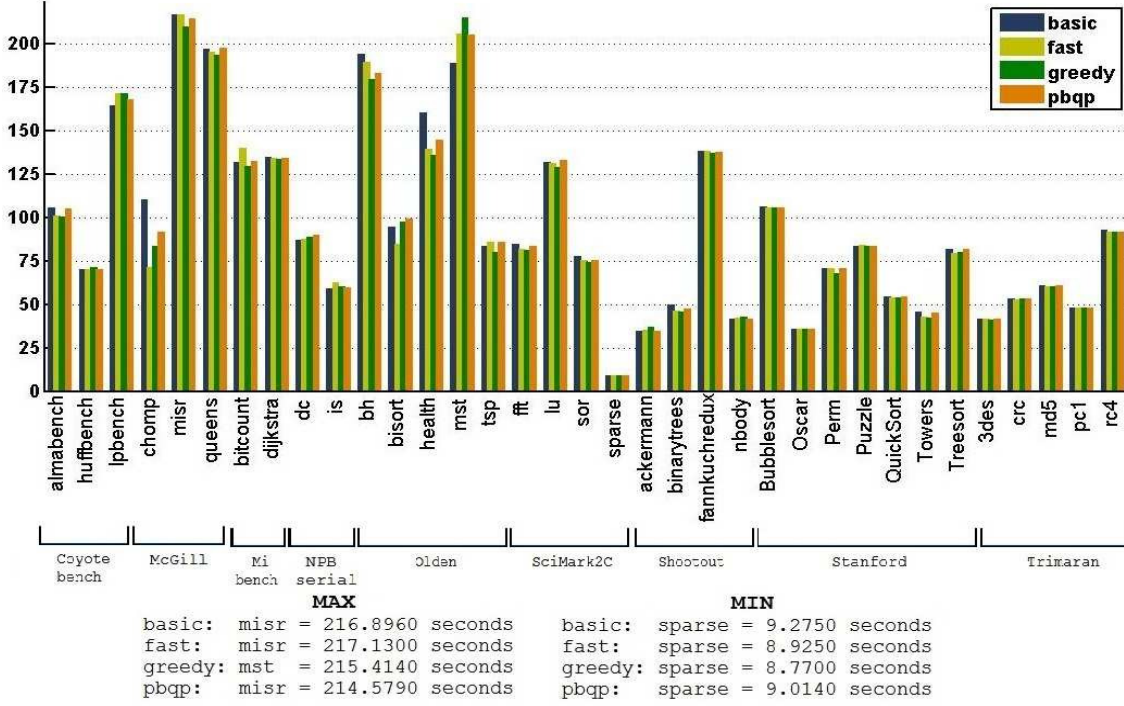


Fig. 1. Runtime in seconds for each program using different allocators

The benchmarks used in the experiments belong to LLVM benchmark test suit.

The experimental evaluation of LLVM allocators was conducted to measure the impact of each allocation strategy from the perspective of three parameters: runtime, compile time and amount of spills. The runtime was collect by system calls like *time*, compile time was obtained through the information available by LLVM, and the amount of spills was obtained directly from the source code. The discussion of the results is also based on information obtained from hardware performance counters that were collected during the program execution using the PAPI tool[42].

A. Runtime

The runtime of each program (measured in seconds) can be seen in Figure 1. Analyzing the runtime is possible to note that *greedy* outperforms the others in 20 programs (*almabench*, *misr*, *queens*, *bitcount*, *dijkstra*, *bh*, *health*, *tsp*, *fft*, *lu*, *sor*, *sparse*, *binarytrees*, *fannkuchredux*, *perm*, *puzzle*, *quicksort*, *towers*, *3des*, and *rc4*). In fact, *basic* was the best only for 5 programs (*huffbench*, *is*, *mst*, *ackerman*, and *oscar*), *fast* only for 7 (*chomp*, *ds*, *bisort*, *bubblesort*, *treedsort*, *crc*, and *md5*), and *PBQP* only for 3 (*lpbench*, *nbody*, and *pc1*).

In general, *greedy* is the best choice for compiling programs from MCGILL, MIBENCH, OLDEN, SCIMARK2C, SHOOTOUT, and STANFORD. *Fast* is the worst choice for compiling COYOTE programs, because this register allocator doesn't outperform the others for any program. The same

situation occurs for *greedy* and *PBQP* in NPBSERIAL, and with *basic* in TRIMARAN. Besides, it is important to note that *greedy* is able to outperform the others in two benchmarks (MIBENCH, and SCIMARK2C) for all programs, and in three benchmarks (MCGILL, OLDEN, and STANFORD) for almost all programs.

The main reason for *greedy* outperforms the others allocators is because it was able to generate the smallest amount of assembly instructions, which it will be discussed in more detail in the Section IV-D. This reduction is occasioned by its ability in eliminating spills.

The Table I shows the relative improvement of each allocator in percentage. In this table the first column presents the program name. The next three columns show the relative improvement of *fast*, *greedy*, and *PBQP* compared with *basic*. The next two columns show the relative improvement of *greedy*, and *PBQP* compared with *fast*. And, the last column shows the relative improvement of *PBQP* compared with *greedy*. Besides, a positive number indicates improvement while negative one indicates performance lost.

The results present in Table I demonstrate that *fast*, *greedy*, and *PBQP* are able to improve the performance up to 35% compared with *basic*. But, in some cases the performance lost can achieve up to 15%. Comparing *greedy*, and *PBQP* with *fast* the improvement can achieve up to 28%, but the performance lost can achieve up to 7.65%. And, for almost all cases *PBQP* was not able to improve the performance of *greedy*.

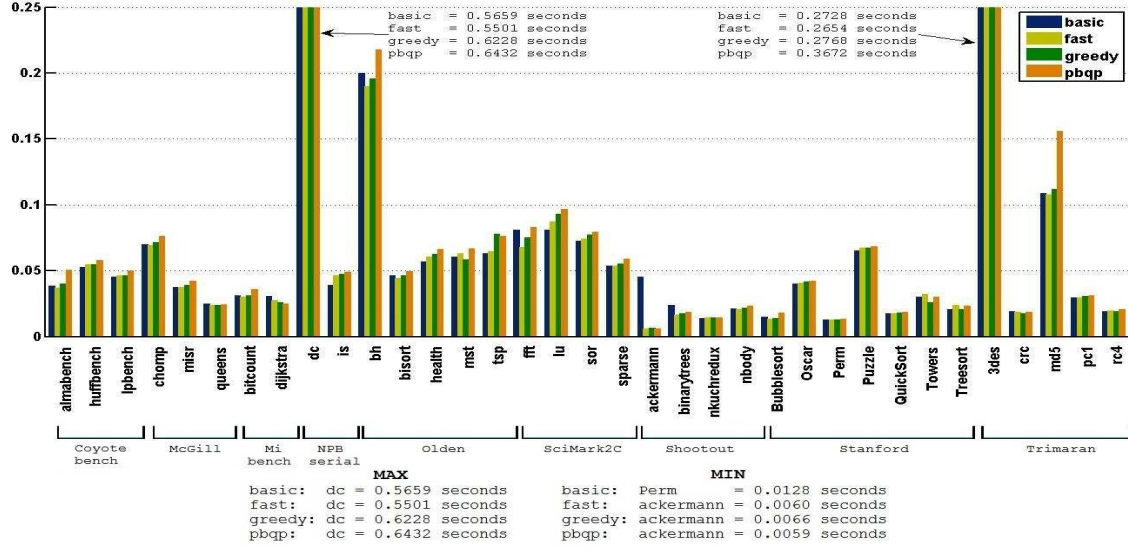


Fig. 2. Compile time in seconds of the LLVM register allocators

TABLE I
THE RELATIVE IMPROVEMENT OF EACH LLVM REGISTER ALLOCATOR
(RUNTIME)

Program	Basic			Fast		Greedy
	Fast	Greedy	PBQP	Greedy	PBQP	
almabench	-4.27	-5.00	-0.20	-0.77	4.25	5.05
huffbench	0.62	2.04	0.01	1.41	-0.60	-1.99
lpbench	4.25	4.19	2.11	-0.06	-2.05	-2.00
chomp	-35.46	-24.11	-16.84	17.58	28.84	9.58
misr	0.11	-3.10	-1.07	-3.20	-1.17	2.09
queens	-0.84	-1.78	0.32	-0.95	1.17	2.14
bitcount	1.12	2.32	3.32	-7.65	5.22	2.64
dijkstra	5.69	1.62	0.88	-0.59	-0.16	0.43
dc	1.12	2.32	3.32	1.19	2.17	0.97
is	5.69	1.62	0.88	-3.85	-4.55	-0.73
bh	-2.44	-7.60	-5.79	-5.29	-3.43	1.96
bisort	-10.69	2.61	4.73	14.89	17.26	2.06
health	-13.21	-15.05	-9.85	-2.12	3.87	6.12
mst	9.08	14.00	8.85	4.51	-0.21	-4.52
tsp	2.73	-4.47	2.50	-7.01	-0.23	7.29
fft	-3.41	-4.40	-1.39	-1.02	2.09	3.15
lu	-0.46	-2.47	0.84	-2.02	1.31	3.40
sor	-2.84	-4.07	-2.57	-1.26	0.28	1.56
sparse	-3.77	-5.44	-2.81	-1.74	1.00	2.78
ackermann	1.54	5.72	-0.20	4.12	-1.72	-5.60
binarytrees	-6.77	-8.35	-4.69	-1.70	2.23	4.00
fannkuch	0.19	-0.54	-0.28	-0.73	-0.47	0.26
nbody	0.32	2.19	-0.94	1.87	-1.25	-3.06
bubblesort	-0.40	-0.17	-0.31	0.22	0.09	-0.13
oscar	0.07	0.10	0.23	0.03	0.16	0.13
perm	0.43	-3.60	-0.10	-4.01	-0.53	3.63
puzzle	0.71	-0.02	0.23	-0.72	-0.48	0.24
quicksort	-1.71	-1.87	-0.26	-0.70	0.92	1.64
towers	-6.61	-7.99	-1.24	-1.47	5.76	7.33
treesort	-2.31	-2.15	0.61	0.17	2.99	2.81
3des	0.41	-0.55	0.48	-0.95	0.07	1.03
crc	-0.20	0.30	0.27	0.50	0.47	-0.03
md5	-0.98	-0.94	-0.37	0.04	0.62	0.58
pc1	0.82	0.07	-0.11	-0.74	-0.92	-0.18
rc4	-1.03	-1.32	-0.90	-0.30	0.13	0.43

B. Compile Time

The results for compile time are shown in Figure 2. Analyzing the compile time is possible to note that *fast* outperforms the others in 19 programs (*almabench*, *chomp*, *misr*, *queens*, *bitcount*, *dc*, *bh*, *bisort*, *fft*, *sparse*, *ackermann*, *binarytrees*, *nbody*, *bubblesort*, *3des*, *crc*, *md5*, *pc1*, and *rc4*), and *basic* in 13 programs (*huffbench*, *lpbench*, *is*, *health*, *tsp*, *lu*, *sor*, *oscar*, *perm*, *puzzle*, *quicksort*, *towers*, and *treesort*). But, *greedy* outperforms the others in only 2 programs (*fannkuchredux*, and *mst*), and PBQP only in 1 (*dijkstra*).

Basic and *fast* are the fastest register allocator due to its simple and fast strategy, which not cause a increase in the compile time. Although, the *greedy* performs a similar strategy to that used by *fast*, the refinement process that consists of splitting the code, spill life interval unions and try reassign intervals, which is repeated continuously, causes a increase in compile time. The PBQP is very expensive because for each function is necessary to create a new assignment problem, solve it and map it to register allocation problem, which requires a lot of time.

The Table II shows the relative improvement of each allocator in percentage. As Table I, in Table II the first column presents the program name. The next three columns show the relative improvement of *fast*, *greedy*, and PBQP compared with *basic*. The next two columns show the relative improvement of *greedy*, and PBQP compared with *fast*. And, the last column shows the relative improvement of PBQP compared with *greedy*. Besides, a positive number indicates improvement while negative one indicates performance lost.

The results present in Table II demonstrate that *basic* and *fast* are able to reduce the compile time up to 30% compared with *greedy*, and PBQP. But, *greedy* and PBQP increase the compile time up to 20% and 38%, respectively.

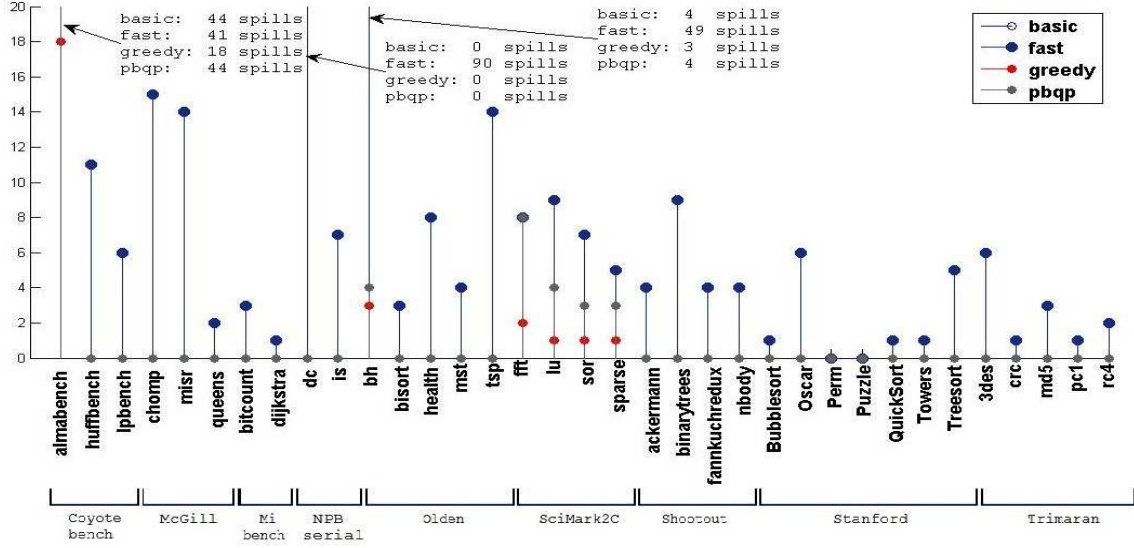


Fig. 3. Amount of spills of the LLVM register allocators

TABLE II
THE RELATIVE IMPROVEMENT OF EACH LLVM REGISTER ALLOCATOR
(COMPILE TIME)

Program	Basic			Fast		Greedy
	Fast	Greedy	PBQP	Greedy	PBQP	PBQP
almanbench	-4.40	3.89	30.83	8.67	36.86	25.94
huffbench	3.21	3.59	9.83	0.37	6.41	6.02
lpbench	2.42	1.76	10.79	-0.65	8.17	8.87
chomp	-0.58	2.57	8.71	3.16	9.34	5.99
misr	-0.27	2.92	11.94	3.19	12.23	8.76
queens	-3.61	-3.61	-1.61	0.00	2.08	2.08
bitcount	-3.21	-0.64	14.42	2.65	18.21	15.16
dijkstra	-10.49	-14.75	-18.03	-4.76	-8.42	-3.85
dc	-2.79	10.05	13.66	13.22	16.92	3.28
is	18.41	21.74	25.32	2.81	5.83	2.94
bh	-5.10	-2.25	8.85	3.00	14.69	11.35
bisort	-4.51	-0.21	6.22	4.49	11.24	6.45
health	6.32	10.00	16.32	3.47	9.41	5.74
mst	3.97	-3.64	10.08	-7.31	5.88	14.24
tsp	2.70	23.81	21.27	20.56	18.08	-2.05
fft	-15.97	-6.68	2.85	11.05	22.39	10.21
lu	7.65	14.57	19.38	6.42	10.89	4.20
sor	1.92	6.32	9.07	4.31	7.01	2.58
sparse	-0.56	2.23	9.09	2.80	9.70	6.72
ackermann	-86.73	-85.40	-86.95	10.00	-1.67	-10.61
binarytrees	-31.38	-27.62	-22.18	5.49	13.41	7.51
fannkuch	2.84	1.42	3.55	-1.38	0.69	2.10
nbody	-0.48	2.38	11.90	2.87	12.44	9.30
bubblesort	-9.46	-5.41	20.95	4.48	33.58	27.86
oscar	0.25	2.98	4.47	2.72	4.21	1.45
perm	0.00	1.56	3.91	1.56	3.91	2.31
puzzle	2.91	3.07	4.60	0.15	1.64	1.49
quicksort	1.14	2.86	5.71	1.69	4.52	2.78
towers	7.33	-14.33	0.33	-20.19	-6.52	17.12
treeseort	16.43	0.48	12.08	-13.69	-3.73	11.54
3des	-2.71	1.47	34.60	4.30	38.36	32.66
crc	-2.63	-6.32	-2.63	-3.78	0.00	3.93
md5	-0.92	2.76	43.47	3.71	44.81	39.62
pc1	-0.34	3.04	5.07	3.39	5.42	1.97
rc4	2.63	1.58	7.89	-1.03	5.13	6.22

C. Spills

Figure 3 shows the amount of spills generated by each register allocator for each benchmark. In this figure, the transparent circles represent the amount of spills to the *basic*, the blue for the *fast*, the red for *greedy* and the ashes to the *PBQP*.

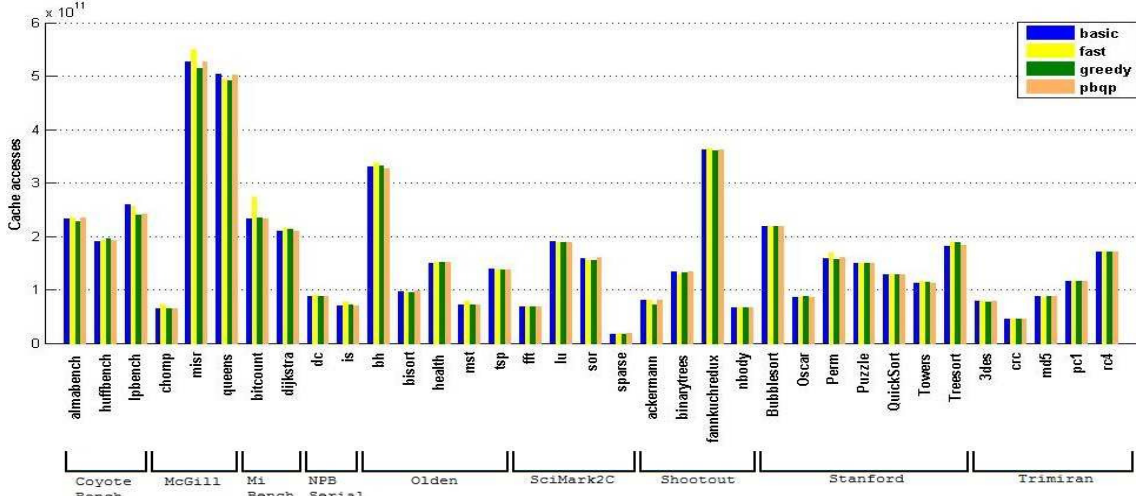
Of the 35 programs evaluated, in 29 the *greedy* allocator didn't generate spills, which demonstrates its best performance in this parameter for most benchmarks. Even in cases where the *greedy* generated spills, it generated the smaller amount. This factor contributes to the good performance of *greedy* on compile time. By having less spills generated, the final code size and the number of memory operations are decreased, allowing faster runtime.

D. Discussion

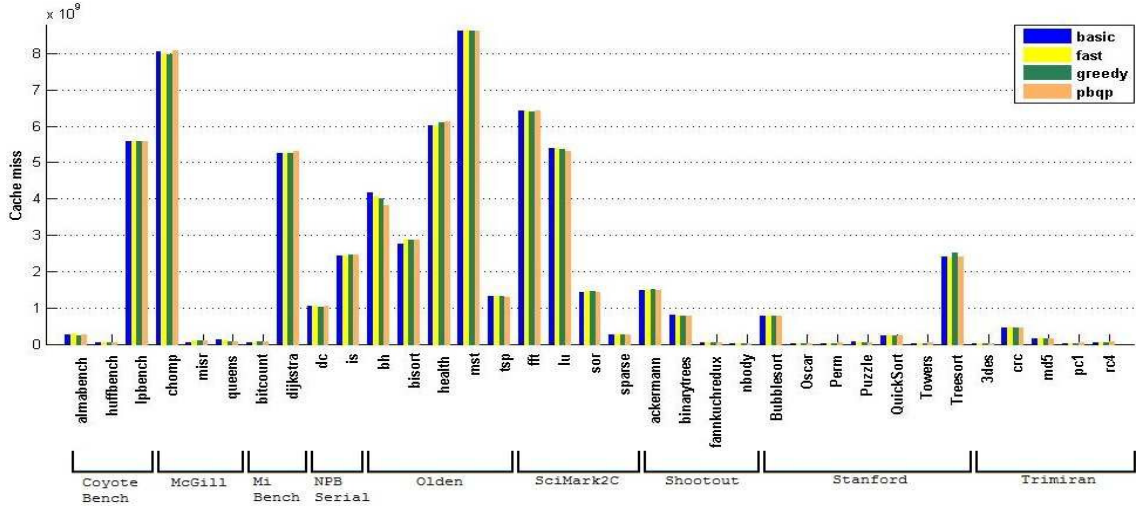
The purpose of this section is to understand the behavior of the final code generated by each allocators that influenced the results presented in the previous sections. For this purpose we collected hardware performance counters during program runtime, namely: cache accesses, cache misses, completed instructions, cycles, and stalls.

Figure 4a shows the amount of cache accesses for each allocator. And, Figure 4b shows the amount of cache misses also for each allocator. In general these amounts influence the performance. In fact, an increase in these amounts tends to cause performance lost due to the extra cycles accessing the memory hierarchy.

Greedy and *basic* are the allocators that obtained the best performance, if we analyze the amount of cache accesses. *Greedy* outperforms the others allocators in 13 programs (*almanbench*, *lpbench*, *chomp*, *misr*, *queens*, *mst*, *tsp*, *lu*, *sparse*, *ackermann*, *fannkuchredux*, *perm*, and *3des*), *basic* also in 13 programs (*huffbench*, *dijkstra*, *dc*, *is*, *health*, *fft*, *nbody*,



(a) Cache accesses of the LLVM register allocators



(b) Cache misses of the LLVM register allocators

Fig. 4. Cache accesses and misses of the LLVM register allocators

bubblesort, *oscar*, *puzzle*, *quicksort*, *treesort*, and *crc*), *fast* in 6 programs (*bisort*, *sort*, *binarytrees*, *md5*, *pc1*, and *rc4*), and PBQP in 3 programs (*bitcount*, *bh*, and *towers*). In average, *greedy* was able to generate 2.33% less cache accesses than *fast*, 0.58% than *basic*, and 0.54% than PBQP. And also in average, *basic* generated 0.51% more cache accesses than *greedy*, 0.02% than PBQP, but 1.77% less cache accesses than *fast*.

The Figure 4b shows that there are a lot of cache misses for NPBSERIAL, OLDEN, and SCIMARK2C in almost all programs. It occurs due to the characteristics of these programs. These results demonstrated that *basic* and *fast* outperform the others, a situation that sounds a little strange if we compared them with previous results. However, we can't analyze only one result, we need to analyze several results. In fact, even though *basic* and *fast* were able to generate

less cache misses than *greedy*, they had more cache access than it. Thus, relatively *greedy* generate less cache misses than *basic*, and *fast*.

The Table III summarizes the average of cache accesses and misses for each allocator. These results explain the good performance of *greedy* register allocator. It was the register allocator that occasioned the least ratio of cache misses. The results showed in this table demonstrated that the use of *greedy* register allocator obtained in average the lowest L1 cache accesses and it increased the amount of L3 cache access. Besides, if we analyse the amount of L2 cache accesses *greedy* also had the worse performance.

Although the *greedy* has obtained the least amount of cache misses in comparison with the other allocators on L3 cache level, the runtime obtained compiling with *greedy* would suggest fewer misses generated by L1 and L2 caches.

TABLE III
CACHE ACCESSES AND CACHE MISSES

PC	Basic	Fast	Greedy	PBQP
# L1 accesses	1.6271e11	1.6557e11	1.6143e11	1.6234e11
# L2 accesses	1.7575e09	1.7588e09	1.7626e09	1.7554e09
# L3 accesses	4.3266e08	4.3283e08	4.3158e08	4.3396e08
# L1 misses	1.2042e09	1.2072e09	1.2103e09	1.2055e09
# L2 misses	4.1910e08	4.1942e08	4.2158e08	4.2065e08
# L3 misses	2.5694e08	2.5148e08	2.4788e08	2.5172e08

These results indicate that for the L1 cache, greedy allocator increased the amount of cache misses. In this scenario, although the greedy register allocator had increased the amount of L1 cache misses, the performance lost was addressed by a smaller amount of cache accesses at these levels. Therefore, the amount of cache miss did not influence significantly the program runtime.

We also investigated the amount of instructions completed, cycles, and stalls. The Table IV summarizes the average amount of the instructions, cycles, and stalls for each LLVM register allocator.

TABLE IV
INSTRUCTIONS, CYCLES, AND STALLS

PC	Basic	Fast	Greedy	PBQP
Instructions	174.6408e11	173.2006e11	169.7774e11	174.8864e11
Cycles	162.7124e9	165.569e9	161.426e9	162.345e9
Stalls	969.983e8	941.754e8	936.860e8	952.324e8

The greedy is the allocator that obtained the best performance. Compared with other allocators, compiling with greedy allocator decreased at least an average 3.4232e09 instructions completed. It is important to note that greedy also outperforms the other allocators in the amount of cycles, and stalls. Thus, these parameters can explain how the code generated by greedy outperforms the code generated by other allocators in runtime.

V. CONCLUDING REMARKS

This work evaluated the four LLVM register allocators. Three main parameters were evaluated: runtime, compile time and the amount of spills. Additionally, hardware performance counters were analyzed to identify the dynamic behavior of the programs compiled with different allocators.

The results demonstrated that for execution time and amount of spills the best allocator is greedy. On the other hand, fast and especially basic if the goal is compile time. These results suggest that for applications requiring short response time or a large amount of processing the greedy register allocator is a good choice. However, with the popularization of virtual machine with JIT compilation, which is critical to generate code quickly, the results suggest the use of the basic or fast.

With respect to hardware performance counters the greedy register allocator, which obtained the better runtime, was able to minimize the amount of cache access, completed instructions, cycles, and stalls. Furthermore, it generated smaller

code than the other allocators. This information indicates the fact that an allocator that reduces the amount of assembly instructions is preferable. It is a important question that can be seen as a guideline to be considered in the design of future register allocators.

REFERENCES

- [1] C. N. Fischer, R. K. Cytron, and R. J. LeBlanc, *Crafting a Compiler*. Addison Wesley, 2010.
- [2] S. S. Muchnick, *Advanced Compiler Design and Implementation*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1997.
- [3] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison Wesley, August 2006.
- [4] W. Stallings, *Computer Organization and Architecture*. Prentice Hall, 2010.
- [5] A. W. Appel, *Modern Compiler Implementation in C*. New York, NY, EUA: Cambridge University Press, 1998.
- [6] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein, "Register Allocation via Coloring," *Computer Languages*, vol. 6, no. 1, pp. 47 – 57, 1981.
- [7] F. M. Q. a. Pereira and J. Palsberg, "Register Allocation via Coloring of Chordal Graphs," in *Proceedings of the Third Asian conference on Programming Languages and Systems*. Berlin, Heidelberg: Springer-Verlag, 2005, pp. 315–329.
- [8] L. George and A. W. Appel, "Iterated Register Coalescing," *ACM Transactions on Programming Languages and Systems*, vol. 18, pp. 300–324, May 1996.
- [9] M. Poletto and V. Sarkar, "Linear Scan Register Allocation," *ACM Transactions on Programming Languages and Systems*, vol. 21, pp. 895–913, September 1999.
- [10] E. Johansson and K. F. Sagonas, "Linear Scan Register Allocation in a High-Performance Erlang Compiler," in *Proceedings of the International Symposium on Practical Aspects of Declarative Languages*. London, UK, UK: Springer-Verlag, 2002, pp. 101–119.
- [11] H. Mössenböck and M. Pfeiffer, "Linear Scan Register Allocation in the Context of SSA Form and Register Constraints," in *Proceedings of the International Conference on Compiler Construction*. London, UK: Springer-Verlag, 2002, pp. 229–246.
- [12] C. Wimmer and H. Mössenböck, "Optimized Interval Splitting in a Linear Scan Register Allocator," in *Proceedings of the ACM/USENIX International Conference on Virtual Execution Environments*. New York, NY, USA: ACM, 2005, pp. 132–141.
- [13] C. N. Lintzmayer, M. H. Mulati, and A. F. da Silva, "Register Allocation by Evolutionary Algorithm," in *XXXI International Conference of the Chilean Computer Science Society*, Valparaíso, Chile, 2012.
- [14] —, "Register Allocation with Graph Coloring by Ant Colony Optimization," in *XXX International Conference of the Chilean Computer Science Society*, Curico, Chile, 2011.
- [15] J. Lidman, D. J. Quinlan, C. Liao, and S. A. McKee, "ROSE::FTTransform - A source-to-source Translation Framework for Exascale Fault-tolerance Research," in *Proceedings of the International Conference on Dependable Systems and Networks Workshops*, 2012, pp. 1–6.
- [16] M. Beck, *Firm an Intermediate Language for Compiler Research*, ser. Interner Bericht. Univ., Fak. für Informatik, Bibliothek, 2005.
- [17] C. Lattner and V. Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation," in *Proceedings of the 2004 International Symposium on Code Generation and Optimization*, Palo Alto, California, Mar 2004.
- [18] M. Arnold, S. Fink, D. Grove, M. Hind, and P. F. Sweeney, "Adaptive Optimization in the Jalapeno JVM," *SIGPLAN Notices*, vol. 46, pp. 65–83, May 2011.
- [19] K. Ishizaki, M. Takeuchi, K. Kawachiya, T. Suganuma, O. Gohda, T. Inagaki, A. Koseki, K. Ogata, M. Kawahito, T. Yasue, T. Ogasawara, T. Onodera, H. Komatsu, and T. Nakatani, "Effectiveness of Cross-platform Optimizations for a Java Just-in-Time Compiler," *SIGPLAN Notices*, vol. 38, pp. 187–204, October 2003.
- [20] T. Suganuma, T. Ogasawara, K. Kawachiya, M. Takeuchi, K. Ishizaki, A. Koseki, T. Inagaki, T. Yasue, M. Kawahito, T. Onodera, H. Komatsu, and T. Nakatani, "Evolution of a Java Just-in-Time Compiler for IA-32 Platforms," *IBM Journal of Research and Development*, vol. 48, pp. 767–795, September 2004.

- [21] D. A. Patterson and J. L. Hennessy, *Computer Organization And Design: The Hardware/software Interface*. The Morgan Kaufmann, 2008.
- [22] F. Glover and M. Laguna, *Tabu Search*. Norwell, MA, USA: Kluwer Academic Publishers, 1997.
- [23] G. J. Chaitin, "Register Allocation & Spilling via Graph Coloring," *SIGPLAN Notices*, vol. 17, pp. 98–101, 1982.
- [24] P. Bergner, P. Dahl, D. Engebretsen, and M. O'Keefe, "Spill Code Minimization via Interference Region Spilling," *SIGPLAN Notices*, vol. 32, pp. 287–295, May 1997.
- [25] D. Bernstein, M. Golumbic, y. Mansour, R. Pinter, D. Goldin, H. Krawczyk, and I. Nahshon, "Spill Code Minimization Techniques for Optimizing Compilers," *SIGPLAN Notices*, vol. 24, pp. 258–263, June 1989.
- [26] P. Briggs, K. D. Cooper, and L. Torczon, "Improvements to Graph Coloring Register Allocation," *ACM Transactions on Programming Languages and Systems*, vol. 16, pp. 428–455, May 1994.
- [27] J. Runeson and S.-O. Nyström, "Retargetable Graph-Coloring Register Allocation for Irregular Architectures," in *Proceedings of the Software and Compilers for Embedded Systems*. Springer, 2003, pp. 22–8.
- [28] J.-M. Daveau, T. Thery, T. Lepley, and M. Santana, "A Retargetable Register Allocation Framework for Embedded Processors," *SIGPLAN Notices*, vol. 39, pp. 202–210, June 2004.
- [29] A. Mahajan and M. S. Ali, "Hybrid Evolutionary Algorithm Based Solution for Register Allocation for Embedded Systems," *Journal of Computers*, vol. 3, no. 6, 2008.
- [30] S. Shamizi and S. Lotfi, "Register Allocation via Graph Coloring using an Evolutionary Algorithm," in *Proceedings of the International Conference on Swarm, Evolutionary, and Memetic Computing - Volume Part II*. Berlin, Heidelberg: Springer-Verlag, 2011, pp. 1–8.
- [31] H. R. Topcuoglu, B. Demiroz, and M. Kandemir, "Solving the Register Allocation Problem for Embedded Systems Using a Hybrid Evolutionary Algorithm," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 5, pp. 620–634, Oct. 2007.
- [32] S. Wu and S. Li, "Extending Traditional Graph-Coloring Register Allocation Exploiting Meta-heuristics for Embedded Systems," in *Proceedings of the Third International Conference on Natural Computation*. Washington, DC, USA: IEEE Computer Society, 2007, pp. 324–329.
- [33] D. Ashlock, *Evolutionary Computation for Modeling and Optimization*. Springer, 2005.
- [34] I. Blöchliger and N. Zufferey, "A Graph Coloring Heuristic using Partial Solutions and a Reactive Tabu Scheme," *Computers & Operations Research*, vol. 35, no. 3, pp. 960–975, 2008.
- [35] A. Hertz and D. Werra, "Using Tabu Search Techniques for Graph Coloring," *Computing*, vol. 39, pp. 345–351, 1987.
- [36] K. D. Cooper, A. Dasgupta, and J. Eckhardt, "Revisiting Graph Coloring Register Allocation: A Study of the Chaitin-Briggs and Callahan-Koblenz Algorithms," in *Proceedings of the International Conference on Languages and Compilers for Parallel Computing*. Berlin, Heidelberg: Springer-Verlag, 2006, pp. 1–16.
- [37] F. M. Quintão Pereira and J. Palsberg, "Register Allocation by Puzzle Solving," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*. New York, NY, USA: ACM, 2008, pp. 216–226.
- [38] Z. Cai, A. Liang, Z. Qi, L. Jiang, X. Li, H. Guan, and Y. Chen, "Performance Comparison of Register Allocation Algorithms in Dynamic Binary Translation," in *Proceedings of the International Conference on Knowledge and Systems Engineering*, oct. 2009, pp. 113 –119.
- [39] V. Adve, C. Lattner, M. Brukman, A. Shukla, and B. Gaeke, "LLVA: A Low-level Virtual Instruction Set Architecture," in *Proceedings of the International Symposium on Microarchitecture*, San Diego, California, Dec 2003.
- [40] L. Hames and B. Scholz, "Nearly optimal register allocation with pbqp," in *In Proceedings of the 7th Joint Modular Languages Conference (JMLC'06). LNCS*. Springer, 2006, pp. 346–361.
- [41] C. W. Commander, "A survey of the quadratic assignment problem, with applications," *Morehead Electronic Journal of Applicable Mathematics*, vol. 4, pp. MATH-2005-01, 2005.
- [42] D. Terpstra, H. Jagode, H. You, and J. Dongarra, "Collecting Performance Data with PAPI-C," in *Tools for High Performance Computing 2009*, M. S. Müller, M. M. Resch, A. Schulz, and W. E. Nagel, Eds. Springer Berlin Heidelberg, 2010, pp. 157–173.